

# Chapter 1 — Practical Task 01: Choosing a Caching Strategy

Road to Architect — Designing Data-Intensive Applications

Chapter 1: Reliable, Scalable and Maintainable Applications

## Where this comes from

“For example, if you have an application-managed caching layer (using memcached or similar) ... it is normally the application code’s responsibility to keep those caches ... in sync with the main database.” — Kleppmann, *DDIA*, Chapter 1

## What you already have right

An application-managed cache means **your code**, not the cache server, decides what to cache and when to invalidate it. That specific pattern has a name: **cache-aside** (a.k.a. lazy-loading). It’s one of several named strategies — the real skill isn’t knowing cache-aside exists, it’s knowing when to reach for a *different* one.

| Strategy                         | How writes work                                                                                          | Staleness risk                              | Write latency added                            | Complexity                                        | Best for                                                      |
|----------------------------------|----------------------------------------------------------------------------------------------------------|---------------------------------------------|------------------------------------------------|---------------------------------------------------|---------------------------------------------------------------|
| <b>Cache-aside</b>               | App writes to DB, then invalidates/deletes the cache key                                                 | Brief, if invalidation is missed or delayed | None (cache untouched on write path)           | Low                                               | Read-heavy, staleness tolerable                               |
| <b>Read-through</b>              | Same as cache-aside, but the <i>cache library</i> handles the miss-then-populate step, not your app code | Same as cache-aside                         | None                                           | Low-medium (depends on library support)           | Read-heavy, want less app-side plumbing                       |
| <b>Write-through</b>             | App writes to cache; cache synchronously writes to DB before returning                                   | None — cache and DB always agree            | Yes — every write pays DB latency              | Medium                                            | Reads must never be stale, moderate write volume              |
| <b>Write-behind (write-back)</b> | App writes to cache; cache asynchronously flushes to DB later                                            | None for cache reads, but DB can lag        | No — write returns as soon as cache is updated | High (needs durable queue/buffer, crash handling) | Very high write throughput, small data-loss window acceptable |

## Part 1 — Build cache-aside on top of your own “database”

Setup (one-time):

```
brew install redis
redis-server --daemonize yes      # starts Redis in the background on localhost:6379
redis-cli ping                    # should print PONG
```

The folder `practical_01_cache/` (next to this PDF) is already set up for you:

- `slow_db.sh` — your stand-in for “the main database”: correct, but slow (it’s a linear scan over a file — that’s the point, treat it as if it had real disk-seek latency). Copied down from `chapter3/simplest_db.sh`.
- `cache.py` — `db_get/db_set` (the subprocess plumbing to talk to `slow_db.sh`) and the redis client setup are already filled in — that’s just wiring, not the concept being tested. The two functions left as `TODO` are `cached_get` and `cached_set`: that’s the actual cache-aside logic, and it’s the only part you need to write:

```
def cached_get(key: str) -> str | None:
    # TODO (the crucial part): implement cache-aside reads.
    # 1. Ask redis for the key.
    # 2. On a hit, print "CACHE HIT" and return it.
    # 3. On a miss, print "CACHE MISS", fall back to db_get, populate redis
    #    with r.set(key, value, ex=CACHE_TTL_SECONDS), then return it.
    pass

def cached_set(key: str, value: str) -> None:
    # TODO (the crucial part): implement cache-aside writes.
    # 1. Write through to the real DB via db_set.
    # 2. Invalidate the cache so the next read is forced to reload: r.delete(key).
    pass
```

`cd` into `practical_01_cache/`, install the client library (`pip install redis`), fill those two functions in yourself — don’t just copy an answer, that defeats the point — then test it:

```
cached_set("123", "amiay-v1")
cached_get("123")    # expect: CACHE MISS, then returns amiay-v1
cached_get("123")    # expect: CACHE HIT, returns amiay-v1
cached_set("123", "amiay-v2")
cached_get("123")    # expect: CACHE MISS (invalidated), returns amiay-v2
```

Now break it on purpose. Comment out the `r.delete(key)` line in `cached_set`, re-run the same sequence, and watch the last `cached_get` return `amiay-v1` — a stale read — even though the DB has `amiay-v2`. This is the literal meaning of “it’s the application’s responsibility to keep the cache in sync with the main database”: nothing enforces it for you, and forgetting one line is enough to silently serve wrong data. Put the `r.delete(key)` back before moving on.

## Part 2 — Pick a strategy under constraints

For each scenario, **write down your choice and a 2-3 sentence justification before reading the answer below it**. Use the table from Part 1.

---

**Scenario A** — Product listing page on an e-commerce site. Read:write ratio is roughly 1000:1. Showing a price that’s 30 seconds stale is a non-issue.

Your answer: \_\_\_\_\_

*Reasoning (write your own answer above before reading this):* **Cache-aside with a TTL.** Reads dominate so heavily that write-path cost is irrelevant — optimize purely for read latency. A short TTL bounds the

staleness window without needing precise invalidation everywhere writes happen (useful if multiple services can write the underlying data).

---

**Scenario B** — A bank balance shown immediately after a deposit. It must reflect the write instantly. Write volume is low (a few per second, system-wide).

Your answer: \_\_\_\_\_

*Reasoning (write your own answer above before reading this):* **Write-through** (or don't cache the write path at all and rely on cache-aside's synchronous invalidation). Since write volume is low, the extra DB round-trip on write is cheap, and you get zero staleness — correctness beats shaving milliseconds here.

---

**Scenario C** — Clickstream/analytics event ingestion at very high volume. Losing a few seconds of events on a crash is acceptable; falling behind on ingest throughput is not.

Your answer: \_\_\_\_\_

*Reasoning (write your own answer above before reading this):* **Write-behind**. The cache (or a queue in front of it) absorbs bursts and a background worker flushes to the DB asynchronously. You're explicitly trading a small durability window for much higher write throughput — the opposite priority from Scenario B.

---

## Part 3 — Map back to real tools

- **Cache-aside / read-through:** this is what “memcached or similar” (the book's phrase) usually means in practice — Redis or Memcached sitting next to your app, with your code (or a thin client library) owning the get-miss-populate logic.
- **Write-through / read-through as a managed feature:** some managed caches do this *for* you instead of leaving it to app code — e.g. Amazon DynamoDB Accelerator (DAX) supports write-through automatically. That's a case where the “keep cache in sync” responsibility moves from your application to the infrastructure layer.
- **Write-behind:** usually built from a durable queue (Kafka, SQS) plus a consumer that flushes to the DB, rather than a single off-the-shelf flag — because you need to think hard about what happens to queued writes if the consumer crashes.

## The general checklist (reusable beyond caching)

When you hit a “which tool/pattern do I use here” moment, ask:

1. What's the read:write ratio?
2. How much staleness can a reader tolerate?
3. Can I afford to lose the most recent write(s) on a crash?
4. Do I want my application code to own the sync logic, or push that onto a managed layer?

Answering these four *before* reaching for a tool name is the actual skill — the tool choice falls out of the answers.